Lab: Provision an Azure Databricks Workspace using Terraform

Description

In this lab, you will use **Terraform** and the latest **AzureRM provider (v5.6.0)** to provision an **Azure Databricks workspace**, and then verify and launch it from the **Azure Portal**.

By the end of this lab, you will have a working Databricks workspace, fully defined as code, that you can open from the portal.

Learning Objectives

By the end of this lab, you will be able to:

1. Write Terraform configuration to provision an **Azure Databricks workspace**.
2. Register the required **Azure Resource Provider** from the provider block.
3. Understand workspace settings such as **SKU (pricing tier)** and the **managed resource group**.
4. Use **outputs** to capture the workspace URL after deployment.
5. Verify and launch the workspace from the **Azure Portal**.
6. Clean up all resources with **terraform destroy**.

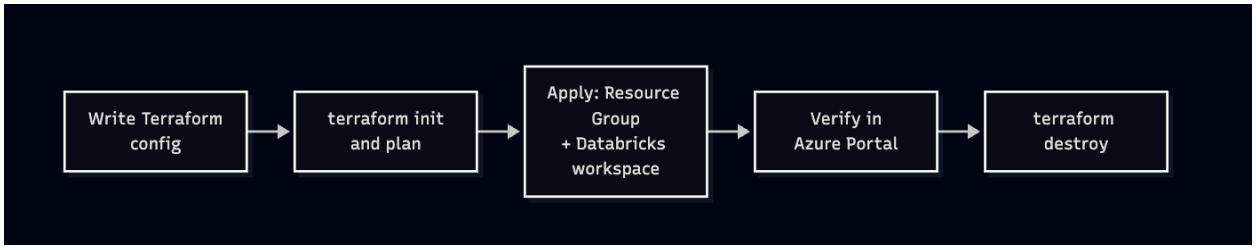
Prerequisites

Make sure you have the following before starting:

- A valid **Azure subscription** where you have **Contributor** (or Owner) access.
- **Terraform** installed locally.
- **Azure CLI** installed locally.
- **Visual Studio Code (VS Code)** or any IDE.

*If you haven't completed these, please go through the previous lab on **Terraform setup**.*

Deployment Flow



Step by Step Guide

Step 1: Authenticate to Azure

Log in with the Azure CLI:

```
az login
```

If you have multiple subscriptions, select the one you want to use:

```
az account set --subscription "<your-subscription-id>"
```

Step 2: Create the Project and Configure the Provider

In VS Code, create a new folder called databricks_workspace. Inside it, create a file named providers.tf:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "5.6.0"
    }
  }
}

provider "azurerm" {
  features {}
  subscription_id = "<your-subscription-id>"

  resource_providers_to_register = [
```

```
    "Microsoft.Databricks",
  ]
}
```

Explanation:

- version = "5.6.0" → pins the latest AzureRM provider release.
- resource_providers_to_register → from v5.0, the provider no longer registers Azure Resource Providers automatically. This line registers Microsoft.Databricks for you.

Initialize the project:

```
terraform init
```

Step 3: Define Variables

Create a file named variables.tf:

```
variable "resource_group_name" {
  type        = string
  description = "Name of the Resource Group"
}

variable "location" {
  type        = string
  description = "Azure region for all resources"
  default     = "West Europe"
}

variable "workspace_name" {
  type        = string
  description = "Name of the Databricks workspace"
}

variable "sku" {
  type        = string
  description = "Pricing tier: standard, premium or trial"
  default     = "premium"
```

```
  validation {
    condition     = contains(["standard", "premium", "trial"], var.sku)
    error_message = "sku must be standard, premium or trial."
  }
}

variable "managed_resource_group_name" {
  type        = string
  description = "Resource group that Azure Databricks manages for you"
}

variable "tags" {
  type        = map(string)
  description = "Tags to assign to resources"
}
```

Step 4: Add Variable Values

Create a file named terraform.tfvars:

```
resource_group_name         = "rg-databricks-lab"
location                    = "West Europe"
workspace_name              = "dbw-terraform-lab"
sku                         = "premium"
managed_resource_group_name = "rg-databricks-lab-managed"
tags = {
  environment = "dev"
  project     = "databricks-lab"
}
```

The managed resource group name must not already exist. Azure creates it for you.

Step 5: Define the Resources

Create a file named resource_group.tf:

```
resource "azurerm_resource_group" "resource_group" {
  name     = var.resource_group_name
  location = var.location
```

```
  tags     = var.tags
}
```

Create a file named databricks_workspace.tf:

```
resource "azurerm_databricks_workspace" "databricks_workspace" {
  name                        = var.workspace_name
  resource_group_name         = azurerm_resource_group.resource_group.name
  location                    = azurerm_resource_group.resource_group.location
  sku                         = var.sku
  managed_resource_group_name = var.managed_resource_group_name

  tags = var.tags
}
```

Explanation:

- sku → the pricing tier. Premium adds role-based access controls and is recommended for teams.
- managed_resource_group_name → Azure places the workspace's storage, network and security resources here.

Create a file named outputs.tf:

```
output "databricks_workspace_url" {
  value = "https://${azurerm_databricks_workspace.databricks_workspace.workspace_url}"
}

output "databricks_workspace_id" {
  value = azurerm_databricks_workspace.databricks_workspace.workspace_id
}
```

Step 6: Deploy the Workspace

Format, validate and preview:

```
terraform fmt
terraform validate
terraform plan
```

Apply the configuration:

```
terraform apply --auto-approve
```

Workspace creation takes about 3 to 5 minutes. When it finishes, view the outputs:

```
terraform output
```

Step 7: Verify in the Azure Portal

1. Log in to the Azure Portal and open **Resource Groups**.
2. Open **rg-databricks-lab** and select **dbw-terraform-lab**.
3. On the **Overview** page, check the **Pricing Tier**, **Workspace URL** and **Managed Resource Group**.
4. Click **Launch Workspace** to sign in to Databricks.
5. Open **rg-databricks-lab-managed** to see the storage account, virtual network and security group Azure created.

Do not modify resources in the managed resource group. Azure Databricks controls them.

Step 8: Clean Up Resources

To avoid unnecessary costs, delete everything created in this lab:

```
terraform destroy --auto-approve
```

This removes the workspace and resource group. Azure deletes the managed resource group automatically. Confirm both resource groups are gone in the Azure Portal.

Troubleshooting

- **MissingSubscriptionRegistration** → check that **resource_providers_to_register** includes **Microsoft.Databricks**, then re-run apply.
- **Managed resource group already exists** → pick a new **managed_resource_group_name** in terraform.tfvars.
- **Unsupported argument** errors → run **terraform init -upgrade** so the 5.6.0 provider is downloaded.

Outcomes

You've provisioned a complete **Azure Databricks environment** using Terraform. You now know how to:

- Provision an **Azure Databricks workspace** with the latest AzureRM provider.
- Register **Azure Resource Providers** directly from the provider block.
- Choose a **pricing tier** and control the **managed resource group** name.
- Verify and launch the workspace from the **Azure Portal**.
- Manage the full lifecycle with **init → plan → apply → destroy**.

This sets the foundation for managing Databricks clusters, jobs and Unity Catalog with Terraform.